

duplicate-logic

October 21, 2025

```
[ ]: # =====  
# DUPLICATE DETECTION STARTUP CONTEXT  
# =====  
  
# Focused only on the system preview relevant to duplicate detection.  
  
# When the SmartCity backend starts:  
#   - FastAPI loads configuration values from config.py:  
#       ID_THRESH (similarity threshold)  
#       DUPE_CHECK_DELAY (delay before comparison)  
#       MIN_POINTS (minimum numeric samples for correlation)  
#  
#   - Core service clients initialize and connect:  
#       → MQTTClient connects to the broker and subscribes to configured topics.  
#       → InfluxDB connection verified for storing incoming numeric data.  
#       → WebSocketManager activated for UI updates.  
#       → TopicHandler and EmbeddingManager prepared for message processing.  
#  
#   - The system listens for incoming MQTT messages.  
#       Each message's topic is checked against embedding_store:  
#           • If topic already known → skip embedding and duplicate scheduling.  
#           • If topic unseen → trigger embedding and duplicate detection.  
#  
#   - Regardless of whether the topic is new or known:  
#       → The message data (topic, tags, values, timestamp) is written into  
#       ↪ InfluxDB.  
#       → The same message is also forwarded via WebSocket for real-time  
#       ↪ visualization.  
#  
'''text  
Startup Flow:  
↓  
FastAPI loads config (ID_THRESH, DUPE_CHECK_DELAY)  
↓  
MQTT, InfluxDB, and WebSocket connections initialized  
↓  
System listening for incoming MQTT messages
```

↓
Each message processed → embedding pipeline triggered
'''

```
[ ]: # =====  
# NEW TOPIC ARRIVAL  
# =====  
  
# A new MQTT message arrives:  
#   topic = "building1/floor2/temp_sensor_A"  
#   payload = { "fields": {"value": 23.4}, "tags": {"floor": "2", "room": "officeA"} }  
#   timestamp = 1732041036  
  
# TopicHandler:  
#   - Extracts topic, tags, and timestamp.  
#   - Checks embedding_store for existing topic:  
#       If topic exists → skip embedding & dupe scheduling.  
#       If new topic → generate embedding and trigger dupe check.  
  
# EmbeddingManager:  
#   - Converts topic + tags to text (flattened string)  
#   - Embeds using BAAI/bge-small-en-v1.5  
#   - Saves embedding to embedding_store.json  
#   - trigger duplicate check logic  
  
'''text  
Topic Arrival Flow:  
↓  
MQTT message received → extract topic and tags  
↓  
Check if topic is new in embedding_store  
↓  
If new → embed + schedule duplicate check  
↓  
If existing → skip embedding  
'''
```

```
[ ]: # =====  
# DUPLICATE CHECK INITIALIZATION  
# =====  
  
# DupeManager.check_new_topic() starts asynchronous background task:  
#   - Waits DUPE_CHECK_DELAY seconds (e.g., 120s)  
#   - Retrieves all topics and embeddings from embedding_store.json  
#   - Begins comparison phase.
```

```

# Comparison is repeated up to 3 times to catch late-arriving topics:
# for retry in [1, 2, 3]:
#     sleep(DUPE_CHECK_DELAY)
#     compare embeddings again
#     if duplicate found → break loop

'''text
Duplicate Check Flow:
↓
DupeManager schedules background comparison task
↓
Wait DUPE_CHECK_DELAY seconds (async)
↓
Fetch all embeddings from embedding_store
↓
Start comparison loop
'''

```

```

[ ]: # =====
# EMBEDDING COMPARISON LOOP
# =====

# for each record in embedding_store:
#     existing_topic = record["topic"]
#     existing_vector = record["embedding"]
#
#     if existing_topic == new_topic:
#         continue # skip self-comparison
#
#     score = DuplicateService.hybrid_score(new_topic, new_emb, existing_topic,
#     ↪existing_emb)
#
#     if score >= ID_THRESHOLD:
#         duplicate_store.add({
#             "topics": [new_topic, existing_topic],
#             "score": score,
#             "status": "PENDING"
#         })
#         ws_manager.broadcast({
#             "event_type": "duplicate",
#             "data": record
#         })
#         log "[DupeManager] Duplicate found between topics."
#         break
#

```

```
'''text
Comparison Loop:
↓
Iterate over all stored topics
↓
Skip self → compute hybrid similarity
↓
If score > ID_THRESH → mark as potential duplicate
↓
Save record + broadcast via WebSocket
'''
```

```
[ ]: # =====
# HYBRID SIMILARITY CALCULATION
# =====

# Each comparison combines two independent similarity signals:
# 1. Semantic cosine similarity (embedding-based)
# 2. Numeric correlation (InfluxDB time-series)

# -----
#           Semantic Similarity
# -----
# cosine = dot(embA, embB) / (||A|| * ||B||)
# → Measures semantic closeness between topic names and tags.

# -----
#           Numeric Correlation
# -----
# - The system queries InfluxDB for recent numeric data:
#   valuesA = query_manager.get_last_points(topicA, limit=100)
#   valuesB = query_manager.get_last_points(topicB, limit=100)
# - Each list contains the latest numeric 'value' field readings.
# - Non-numeric entries are filtered out before correlation.

# -----
#           Correlation and Hybrid Score
# -----
# if len(valuesA) and len(valuesB) >= MIN_POINTS:
#     r = pearson_correlation(valuesA, valuesB)
#     corr_score = (r + 1) / 2 # normalize [-1, 1] → [0, 1]
#     weight = min(1.0, N / 100)
#     hybrid_score = (1 - weight)*cosine + weight*corr_score
# else:
#     hybrid_score = cosine # fallback if insufficient data
# return hybrid_score
```

```

'''text
Hybrid Similarity Process:
↓
Query recent numeric readings from InfluxDB for both topics
↓
Compute cosine (semantic) + correlation (numeric)
↓
Blend using dynamic weight based on data availability
↓
Return final hybrid similarity score
'''

```

```

[ ]: # =====
# DUPLICATE DETECTION DECISION
# =====

# if hybrid_score >= ID_THRESH:
#     create record:
#     {
#         "topics": [topic_A, topic_B],
#         "score": hybrid_score,
#         "status": "PENDING"
#     }
#     save to duplicate_store.json
#     notify frontend via WebSocket
#     break # only first strong match triggers
#
# else:
#     continue until all topics checked

'''text
Decision Logic:
↓
Compare hybrid_score with ID_THRESH from config
↓
If above threshold → add to duplicate_store.json
↓
If below → continue checking other topics
↓
Stop when first strong duplicate found
'''

```

```

[ ]: # =====
# USER CONFIRMATION (FRONTEND + BACKEND)
# =====

```

```

# UI shows detected pair:
#   "Duplicate detected: topic_A  topic_B (score 0.93)"
#
# User clicks:
#   "Approve" → confirms duplicate
#   "Reject" → marks not duplicate

# Frontend sends POST request:
#   /confirm-duplicate
#   { "topicA": "temp_sensor_A", "topicB": "temp_sensor_B", "action": "approve"
  ↪ / "delete" }

# Backend:
#   if action == "approve":
#       mark record as "CONFIRMED_DUPLICATE"
#       topic_manager.unsubscribe(topic_B)
#   elif action == "delete":
#       mark as "NOT_DUPLICATE"
#   save store + broadcast update

'''text
User Confirmation Flow:
↓
Frontend receives duplicate pair (status=PENDING)
↓
User approves or rejects in UI
↓
POST /confirm-duplicate → backend
↓
Backend updates duplicate_store + broadcasts change
'''

```

```

[ ]: # =====
# CONTINUOUS MONITORING LOOP
# =====

'''
MQTT Message
↓
EmbeddingManager → DupeManager.check_new_topic()
↓
Async delay (DUPE_CHECK_DELAY)
↓
Loop through embeddings → compute hybrid score
↓
If score > threshold → store duplicate → broadcast UI
↓
'''

```

User approves/rejects + update store + unsubscribe

↓

WebSocket refreshes UI instantly

'''

The system maintains a real-time, self-correcting duplicate detection cycle.